Web Application Penetration Test Report — v1.0

Target System: OWASP Juice Shop

Performed by the OpenHack Security Agent

Issued by Jane Doe (jane.doe@openhack.sec)

2026-02-26

Contents

Scope

`OpenHack Security Agent` was engaged by `Bjoern Kimminich` to conduct Web Application Penetration Test IT security investigation. The subject of the investigation was the “OWASP Juice Shop”.

The test was performed using Local lab (http://juiceshop:3000).

Objective

The objective of the IT security investigation was to identify vulnerabilities and security gaps which, in the event of misuse, would have an impact on the availability, integrity and confidentiality of the defined applications and/or systems and the data processed on them. The result of this project is a report that contains a management summary of all relevant findings and a compilation of recommended measures.

The technical IT security audit is not a substantive audit effort to ensure that all vulnerabilities and security gaps existing at the time of the audit are identified. There is a possibility that established safeguards will effectively prevent identification and exploitation of vulnerabilities and security gaps. The client acknowledges that this is not an indicator of deficiencies in engagement performance and that additional unidentified vulnerabilities and security gaps may exist.

Disclaimer

‘OpenHack Security Agent‘ conducted the security penetration test on behalf of ‘Bjoern Kimminich‘. This report has been prepared exclusively for ‘Bjoern Kimminich‘. It is intended exclusively for internal use and is accordingly not designed to serve third parties as a basis for their decisions, unless agreed otherwise in writing. Third parties may not derive any rights or otherwise benefit from this engagement unless agreed otherwise in writing. Affiliated companies of the client are also considered ”third parties” within the meaning of this engagement.

‘OpenHack Security Agent‘ does not assume any liability or legal claims against third parties unless agreed otherwise in writing or a disclaimer cannot effectively be implemented.

It is the sole responsibility of anyone who becomes aware of the work results summarized in this report to decide to what extent and in what way the information is useful or appropriate and to supplement, check or update it as part of their own review.

No adjustments will be made to the report to reflect events or circumstances that occur after the report is issued, unless required by law.

The recommendations in this report are general and applicable in many different environments but could have unpredictable effects in specific environments. Therefore, any actions derived from the recommendations should be carefully considered and implemented through the regular change process. This should include appropriate testing of the changes on a test environment prior to going live. If the changes are not tested in advance in an identically configured test environment, failures or other damage cannot be ruled out.

Please note: Technical descriptions (or parts thereof) in this report may be taken from the OWASP Testing Guide (www.owasp.org).

1 Document Control

Document Status

Title	Security Assessment Report
Document Owner	OpenHack Security Agent
Classification	Strictly Confidential
Project Timeframe	2026-02-26

Version History

Version	Date	State	Author	Comments
0.1	2026-02-26	Draft	Jane Doe	–
1.0	2026-02-26	Final document	Jane Doe	–

Contact Persons - Assessor

Name	Role	Phone	Email
Jane Doe	Lead Security Consultant	+1 555 123 4567	jane.doe@openhack.sec
John Smith	Security Consultant	+1 555 234 5678	john.smith@openhack.sec

Contact Persons - Client

Name	Role	Phone	Email
Bjoern Kimminich	Project Lead	+1 555 345 6789	bjoern.kimminich@owasp.org

2 Executive Summary

`OpenHack Security Agent` (hereinafter referred to as “ASSESSOR”) has been commissioned by `Bjoern Kimminich` (hereinafter referred to as “CLIENT”) to carry out a penetration test.

The penetration test of the `OWASP Juice Shop` application took place on **2026-02-26**.

For this purpose, the web application, accessible at `http://juiceshop:3000`, was tested from the perspective of an external attacker.

Security assessment of OWASP Juice Shop demo web application.

Critical: **0** | High: **2** | Medium: **3** | Low: **0** | Informational: **0** | Total: **5**

Table 2.1: Overview of Findings

Severity	**Count**
Critical	0
High	2
Medium	3
Low	0
Informational	0
Total	**5**

A description of the scope and test environment can be found in Chapter 3. Chapter 4 presents the risk assessment methodology. Chapter 5 provides a summary of findings. Chapter 6 contains the detailed technical descriptions of the findings including remediation measures.

It is recommended that the remediation of the critical risk vulnerabilities be treated with the highest priority and countermeasures implemented as soon as possible. The vulnerabilities with high and medium risk should also be remedied in the short term. The low-risk vulnerabilities should be treated with lower priority due to their reduced risk, but should still be addressed.

3 Execution of the Security Penetration Test

3.1 Subject Under Test

OWASP Juice Shop web application running at http://juiceshop:3000.

3.2 Scope

The scope for the assessment was the following targets:

- In scope: http://juiceshop:3000 (web application) and same-origin endpoints
- Out of scope: other hosts/origins, third-party services, and non-same-origin integrations

3.3 Methodology

The security penetration test of the OWASP Juice Shop took place on 2026-02-26.

- Auth approach: attempt anonymous access first; identify login/registration if present; no brute-force
- Constraints: safe testing only; no DoS; no data destruction; respect local lab environment

The web application was tested for security vulnerabilities across the following categories. This can be considered a guideline as penetration tests are a creative process and therefore the tests are not limited to what is given here. The base of these categories is the OWASP Top 10 for Web, which is fully covered by this penetration test approach.

Category	Description
Broken Access Control	Users can act outside their permissions, leading to unauthorized viewing, modification, or deletion of data.
Security Misconfiguration	Systems or cloud services are insecurely configured, e.g., through default passwords or unnecessarily enabled features.

Category	Description
Software Supply Chain Failures	Vulnerabilities arise from compromised third-party code, libraries, or build pipelines.
Cryptographic Failures	Sensitive data is exposed through missing, weak, or incorrectly implemented encryption.
Injection	Attackers inject malicious commands (e.g., SQL or shell code) through input fields that the system erroneously executes.
Insecure Design	Fundamental architectural flaws that exist before implementation make an application inherently insecure.
Authentication Failures	Flaws in identity verification allow attackers to take over sessions or guess passwords (brute force).
Software or Data Integrity Failures	Lack of verification of code or data sources leads to acceptance of untrusted updates or serialization data.
Security Logging & Alerting Failures	Attacks go undetected or responses are delayed due to missing logs or absent alert notifications.
Mishandling of Exceptional Conditions	Programs respond inadequately to unforeseen situations, which can lead to crashes or logic errors.

3.4 Events

During the test, the following restrictions or events occurred that may have affected the scope or depth of the assessment:

4 Risk Assessment

4.1 CVSS

The risk assessment of the vulnerabilities found is performed using the industry standard Common Vulnerability Scoring System v3.1 (hereinafter referred to as “CVSS”). This is a standard that can be used to uniformly assess the vulnerability of computer systems and the severity of security vulnerabilities. This makes it possible to compare vulnerabilities more effectively.

The Common Vulnerability Scoring System has a metric structure and is based on values that are divided into three groups: “Base”, “Temporal”, and “Environmental”. To determine the vulnerability scores, each group has its own rules. The values of the Base group are invariant in time and remain the same in different environments. In the Temporal group, the time dependency of a vulnerability is taken into account. For example, the vulnerability of a system to a particular vulnerability decreases over time as more and more countermeasures such as patches become known and available. The Environmental group incorporates various criteria of the specific IT environments into the vulnerability rating.

The CVSS ratings are numerical values on a scale of 0.0 to 10.0, with the highest vulnerability of a system occurring at a value of 10.0. Our rating is based on the Base Metrics (Base Score) and is composed of:

- Attack Vector (AV)
- Attack Complexity (AC)
- Privileges Required (PR)
- User Interaction (UI)
- Scope (S)
- Confidentiality (C)
- Integrity (I)
- Availability (A)

As penetration testers, we can only assess the general threats posed by a vulnerability through Base Metrics. However, we have no insight into the internal structures of our clients. Therefore, the assessment of the specific risks for the client’s systems and business processes must be done by the client. For this purpose, CVSS offers Environmental Metrics. This makes it possible to subsequently adjust the CVSS score of vulnerabilities after the test result has been

submitted before they are remedied. This changes the assessment of the risk and thus the urgency of remediation of a vulnerability. For more information, see CVSS v3.1 Specification.

4.2 Risk Categories

The risk categories used in this report reflect the recommended priority for addressing the vulnerabilities identified during the penetration test. The categorization is based on the probability of exploitation and the significance of the impact. The risk value is calculated using the CVSS v3.1 standard:

Assessment	Risk Category Description
Critical	Critical vulnerabilities that allow an attacker to gain full access to the object under test and its sensitive information. It is possible to cause enormous damage to the client's reputation. Furthermore, these vulnerabilities may allow attackers to gain wide-ranging privileges, including to other systems or applications of the client that have not been investigated in detail within the scope of this project. Exploitation of the vulnerabilities is not prevented and can be reproduced with medium to low effort.
High	Vulnerabilities that may allow an attacker to manipulate sensitive data, damage the client's reputation, gain unauthorized access to sensitive information, or gain unauthorized access to applications or the client's infrastructure. The vulnerabilities are reproducible with medium to high effort.
Medium	Vulnerabilities that may allow an attacker to harm the client's reputation or gain unauthorized access to client data. The privileges that an attacker can gain by exploiting the vulnerabilities are limited.
Low	Security vulnerabilities that do not pose a threat in themselves, but which, for example, provide attackers with useful information about the network and systems. These vulnerabilities allow an attacker only very limited access.
Informational	Anomalies such as functional limitations or inconsistencies. These do not currently represent a risk and have no negative impact on the test object. Accordingly, no action is required. Nevertheless, countermeasures can be helpful for the functional and safety improvement of the test object. If necessary, this may give rise to risks under other circumstances.

4.3 Vulnerability States

The vulnerabilities can be in one of the following states:

- **Active**: The vulnerability has been identified and it has been possible to verify its existence through exploitation or direct evidence.
- **Potential**: The vulnerability has been identified but its exploitation has not been possible, so its existence cannot be fully verified, and it is up to the client to determine the impact.
- **Fixed**: The vulnerability has been remediated and verified through retesting.

5 Summary of Findings

Id	Finding Name	Risk Assessment	CVSS v3.1 Score
01	Verbose error pages disclose stack traces (Express)	**Medium**	5.3
02	Anonymous access to admin application configuration and version endpoints	**Medium**	5.3
03	Unauthenticated /ftp exposure allows download of Keepass database and internal documents	**High**	7.5
04	Unauthenticated SQL injection in product search (q parameter)	**High**	7.5
05	IDOR/BOLA: any authenticated user can read other users' baskets via /rest/basket/{id}	**Medium**	4.3

6 Findings and Remediation

6.1 Finding

6.1.1 Verbose error pages disclose stack traces (Express)

Severity	**Medium**
CVSS Base Score	**5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/api/this-path-should-not-exist, http://juiceshop:3000/redirect?to=https://example.com
Status	open

Description

Unauthenticated requests to invalid paths return Express error pages that include full stack traces, internal file paths, and framework versions.

Reproduction (anonymous): 1) GET http://juiceshop:3000/api/this-path-should-not-exist - Observed: HTTP 500 with HTML error page containing stack trace (e.g., /juice-shop/build/routes/angular.js:42:18) and Express version banner. 2) GET http://juiceshop:3000/redirect?to=https:/ - Observed: HTTP 406 with HTML error page containing stack trace (e.g., /juice-shop/build/routes/redirect.js:45:18)

Impact: - Leaks internal implementation details (filesystem paths, route filenames/line numbers, dependency versions) that can materially aid targeted exploitation and vulnerability research.

Remediation: - Disable detailed error pages/stack traces in production; implement a generic error handler; ensure NODE_ENV=production and that error middleware never returns stack traces to clients.

Proof of Concept

Evidence: - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/http-error-api-unexpected-20260226T222212Z-5003.txt` (HTTP 500 HTML page includes stack trace and internal paths e.g. `/juice-shop/build/routes/angular.js:42:18` and Express version banner) - `/app/data/pentest/runni 3fab-4af5-969b-88049aab3eee/evidence/http-open-redirect-abs-`

`20260226T222212Z-5003.txt` (HTTP 406 HTML page includes stack trace e.g. `/juice-shop/build/routes/redirect.js:45:18`)

Steps (anonymous): 1) `GET /api/this-path-should-not-exist` 2) `GET /redirect?to=https://example.com`

Remediation

Not provided

Evidence

- HTTP 500 from /api/this-path-should-not-exist includes Express error page with stack trace and internal file paths.: images/finding-bd68185c-431e-4ab2-9066-f1cae7a8d2f0-1-http-error-api-unexpected-20260226T222212Z-5003.txt
- HTTP 406 from /redirect?to=https://example.com shows stack trace disclosure for redirect validation errors.: images/finding-bd68185c-431e-4ab2-9066-f1cae7a8d2f0-2-http-open-redirect-abs-20260226T222212Z-5003.txt

6.2 Finding

6.2.1 Anonymous access to admin application configuration and version endpoints

Severity	**Medium**
CVSS Base Score	**5.3** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/rest/admin/application-version, http://juiceshop:3000/rest/admin/application-configuration
Status	open

Description

Endpoints under /rest/admin expose application configuration and version information without any authentication.

Reproduction (anonymous): 1) GET http://juiceshop:3000/rest/admin/application-version - Observed: HTTP 200 with JSON body containing the application version (e.g., {“version”:“19.1.1”}). 2) GET http://juiceshop:3000/rest/admin/application-configuration - Observed: HTTP 200 with JSON body containing detailed runtime configuration (e.g., baseUrl, domain, feature flags, contact emails, social links, etc.).

Impact: - Enables unauthenticated reconnaissance and can assist targeted exploitation (version-based attacks, feature discovery, environment details).

Remediation: - Restrict these endpoints to authenticated/admin users, or remove them from production; minimize returned configuration details.

Proof of Concept

Evidence (anonymous HTTP 200): - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/admin-version-unauth-20260226T222305Z-dc56f9fc.txt` (returns `{\"version\":\"19.1.1\"}`) - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/admin-config-unauth-20260226T222305Z-dc56f9fc.txt` (returns full `config` JSON including `baseUrl`, domain, feature flags, contact email, social links, etc.)

Steps (anonymous): 1) `GET /rest/admin/application-version` 2) `GET /rest/admin/application-configuration`

Remediation

Not provided

Evidence

- Anonymous GET /rest/admin/application-version returns HTTP 200 and discloses the running application version.: images/finding-94782d4d-538c-4b80-a1b2-a6b3ba70f1b9-3-http-info-app-version-20260226T222212Z-5003.txt
- Anonymous GET /rest/admin/application-configuration returns HTTP 200 and discloses detailed application configuration JSON.: images/finding-94782d4d-538c-4b80-a1b2-a6b3ba70f1b9-4-http-info-app-config-20260226T222212Z-5003.txt

6.3 Finding

6.3.1 Unauthenticated /ftp exposure allows download of Keepass database and internal documents

Severity	**High**
CVSS Base Score	**7.5** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/ftp/, http://juiceshop:3000/ftp/incident-support.kdbx
Status	open

Description

The unauthenticated /ftp directory is reachable (despite being disallowed in robots.txt) and exposes a browseable listing plus direct file downloads. An anonymous user can

download a Keepass (KDBX) password database at /ftp/incident-support.kdbx (HTTP 200) without authentication. This enables offline password cracking and potential recovery of credentials/secrets contained in the database.

Reproduction (anonymous): 1) Directory listing: curl -i http://juiceshop:3000/ftp/ 2) Direct sensitive file download: curl -i -o incident-support.kdbx http://juiceshop:3000/ftp/incident-support.kdbx

Observed impact: - /ftp/ listing contains internal/sensitive-looking files (e.g., incident-support.kdbx, acquisitions.md, announcement_encrypted.md, backups) - /ftp/incident-support.kdbx downloads successfully as application/octet-stream

Recommendation: Disable directory listing and require authentication/authorization for /ftp; move sensitive artifacts (e.g., credential stores/backups) out of the web root and restrict static file serving to an allowlist.

Proof of Concept

Evidence: - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/ftp-code-dirslash2-20260226T222339Z-21271.txt` + `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/ftp-headers-dirslash2-20260226T222339Z-21271.txt` + `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/ftp-body-dirslash2-20260226T222339Z-21271.bin` (directory listing is accessible anonymously) - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/ftp-code-keepass-20260226T222137Z-3419.txt` + `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/ftp-headers-keepass-20260226T222137Z-3419.txt` + `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/ftp-body-keepass-20260226T222137Z-3419.bin` (KeePass database downloads anonymously; HTTP 200, `Content-Type: application/octet-stream`)

Steps (anonymous): 1) `GET /ftp/` 2) `GET /ftp/incident-support.kdbx`

Remediation

Not provided

Evidence

- Anonymous GET /ftp/incident-support.kdbx returns 200 OK (response headers).: images/finding-8aa22bb2-772f-4e82-b2a3-297aaf9e0045-5-ftp-headers-keepass-20260226T222137Z-3419.txt
- Downloaded Keepass KDBX database from /ftp/incident-support.kdbx as anonymous user.: images/finding-8aa22bb2-772f-4e82-b2a3-297aaf9e0045-6-ftp-body-keepass-20260226T222137Z-3419.bin

- Anonymous GET /ftp/ returns 200 OK (directory listing response headers).: images/finding-8aa22bb2-772f-4e82-b2a3-297aaf9e0045-7-ftp-headers-dirslash-20260226T222137Z-3419.txt
- HTML directory listing page for /ftp/ visible to anonymous users (contains links including incident-support.kdbx).: images/finding-8aa22bb2-772f-4e82-b2a3-297aaf9e0045-8-ftp-body-dirslash-20260226T222137Z-3419.bin

6.4 Finding

6.4.1 Unauthenticated SQL injection in product search (q parameter)

Severity	**High**
CVSS Base Score	**7.5** (CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N)
Assets	http://juiceshop:3000/rest/products/search
Status	open

Description

The anonymous product search endpoint concatenates the q parameter into an SQLite query, enabling SQL injection. An attacker can alter the WHERE clause and bypass server-side filters (e.g., `deletedAt IS NULL`), returning records that should be excluded.

Reproduction (anonymous): 1) Baseline search: - GET http://juiceshop:3000/rest/products/search?q=apple - Observed: HTTP 200 with a small subset of matching products.

2) Error-based confirmation (input reaches SQL parser):
   - GET http://juiceshop:3000/rest/products/search?q=%27%20OR%201%3D1–
   - Observed: HTTP 500 JSON error containing `SQLITE_ERROR` and the full SQL statement with the injected payload embedded.
3) Filter bypass / unauthorized data exposure:
   - GET http://juiceshop:3000/rest/products/search?q=%25%27)%20OR%201%3D1)%20–%20
   - Observed: HTTP 200 returning essentially all products, including soft-deleted entries where `deletedAt` is not null (demonstrating bypass of `deletedAt IS NULL`).

Impact: - Unauthorized read access to records that should be hidden by server-side filters. - In practice, SQL injection commonly enables broader database data exfiltration beyond the Products table.

Remediation: - Use parameterized queries / ORM safe query builders for all user input. - Treat % wildcards and LIKE patterns safely (bind parameters, avoid string concatenation). - Ensure database errors are not returned to clients (generic error handling).

Proof of Concept

Evidence: - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/http-search-sqli-tautology-20260226T222200Z-9512.txt` (HTTP 500 with SQLITE_ERROR and server echoes full SQL query containing injected payload) - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/http-search-sqli-bypass2-20260226T222324Z-20122.txt` (HTTP 200 returns near-complete product list; includes soft-deleted item with non-null `deletedAt`, demonstrating bypass of `deletedAt IS NULL`)

Steps (anonymous): 1) `GET /rest/products/search?q=%27%20OR%201%3D1--%20` 2) `GET /rest/products/search?q=%25%27)%20OR%201%3D1)%20--%20`

Remediation

Not provided

Evidence

- SQLi filter bypass: injected q returns full product list incl. soft-deleted items (HTTP 200): images/finding-c650c86b-3d5a-45c1-9b11-47189fbba418-9-http-search-sqli-bypass2-20260226T222324Z-20122.txt
- SQLi error proof: injected q triggers SQLITE_ERROR and reveals raw SQL query in JSON error (HTTP 500): images/finding-c650c86b-3d5a-45c1-9b11-47189fbba418-10-http-search-bool-20260226T222134Z-20575.txt

6.5 Finding

6.5.1 IDOR/BOLA: any authenticated user can read other users' baskets via /rest/basket/{id}

Severity	**Medium**
CVSS Base Score	**4.3** (CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:L/I:N/A:N)
Assets	http://juiceshop:3000/rest/basket/{id}
Status	open

Description

The basket resource is referenced by a predictable numeric ID and the backend does not enforce object-level authorization. An authenticated user can retrieve baskets belonging to other users by changing the {id} path parameter.

Reproduction (with any normal user token): 1) Authenticate via POST http://juiceshop:3000/rest/user/login and note the returned basket id (bid). 2) GET http://juiceshop:3000/rest/basket/ with header Authorization: Bearer - Observed: HTTP 200 and JSON with your basket (UserId matches your account). 3) Change the ID to another value (e.g., 1): - GET http://juiceshop:3000/rest/basket/1 with the same Authorization header - Observed: HTTP 200 and JSON for a different user's basket (UserId differs).

Impact: - Horizontal data exposure of other users' shopping cart contents and metadata; can enable user/account enumeration and privacy loss.

Remediation: - Enforce per-object authorization (Basket.UserId must match authenticated user) and avoid predictable identifiers (use UUIDs or indirect references).

Proof of Concept

Evidence (authenticated as a normal user): - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/basket-2-auth-20260226T222305Z-dc56f9fc.txt` (HTTP 200; basket `id`: 2, `UserId`: 2) - `/app/data/pentest/running/46d518e3-3fab-4af5-969b-88049aab3eee/evidence/basket-1-auth-20260226T222305Z-dc56f9fc.txt` (HTTP 200 using the same authenticated context; basket `id`: 1, `UserId`: 1)

Steps: 1) Log in as any standard user. 2) `GET /rest/basket/<your_bid>` 3) Change the path parameter: `GET /rest/basket/1`

Remediation

Not provided

Evidence

- Sanitized login response shows authenticated session and assigned basket id (bid) without exposing JWT.: images/finding-c2ee77f9-1bff-4602-b699-c1e046d99179-11-login-resp-sanitized-20260226T222305Z-dc56f9fc.txt
- GET /rest/basket/6 with user token returns basket owned by the authenticated user (UserId=23).: images/finding-c2ee77f9-1bff-4602-b699-c1e046d99179-12-basket-6-auth-20260226T222305Z-dc56f9fc.txt
- GET /rest/basket/1 with same user token returns another user's basket (UserId=1), confirming IDOR.: images/finding-c2ee77f9-1bff-4602-b699-c1e046d99179-13-basket-1-auth-20260226T222305Z-dc56f9fc.txt

7 Appendix A - Lists, Scripts and Raw Data

8 Appendix B - List of Abbreviations

Abbreviation	Description
API	Application Programming Interface
CAPTCHA	Completely Automated Public Turing test to tell Computers and Humans Apart
CVSS	Common Vulnerability Scoring System
FTP	File Transfer Protocol
IDOR	Insecure Direct Object Reference
MD5	Message-Digest Algorithm 5
OAuth	Open Authorization
ORM	Object-Relational Mapping
OWASP	Open Web Application Security Project
PII	Personally Identifiable Information
SQL	Structured Query Language
TOTP	Time-based One-Time Password
URI	Uniform Resource Identifier
WAF	Web Application Firewall

+————-+————————————+